Structural Certificates for Prediction Depth

Controlled Rules and Sparse Corrections in Cellular Automata

Amamiya Miu

Abstract. We study two sources of shallow circuits for cellular automaton prediction from explicit initial windows. For one-sided frozen controls, a Lean-verified classifier selects a constant-depth, logarithmic-depth, or Pascal branch. A single logspace-uniform family realizes the selected bound at every horizon. In the Pascal branch the minimum De Morgan depth is $2p(T) + 1$ for $T > 0$, with $p(T)$ the binary weight of T. Transported and additive controls illustrate how changes to the rule affect its predictor. For an individual trajectory, we certify a small residual relative to a fixed linear reference by reconstructing the full cone and checking each update. Parallel enumeration yields logspace-uniform NC^1 for constant residual weight or logarithmically many allowed event positions. Chronological repair uses polynomial size with an exponent independent of the correction budget and also handles a bounded number of nonzero update layers. A finite-state coefficient construction supplies the shallow forced response for every fixed homogeneous binary linear reference of finite radius.

Keywords. Cellular automata, circuit depth, logspace uniformity, Lean 4.

1 Introduction

Which verifiable conditions on an evolution rule F and an observation O bound the circuit depth of $O\big(F^{T(x)}\big)$ for a specified input encoding and circuit model? Algebraic structure can make cellular automaton prediction much shallower than layer-by-layer simulation [1]. Here we ask which fixed guards in a one-sided, two-track automaton admit constant-depth prediction, and when a trajectory can be predicted from a small set of discrepancies with a fixed linear rule.

The first result is the four-way classification of frozen rules (Theorem 2). A finite guard test selects the branch, and one logspace generator for the chosen branch achieves the stated upper bound at every time. The main construction uses the forced order of hops to summarize all paths to an endpoint by two prefix statistics. A binary no-carry test then detects which coefficients survive cancellation (Section 3.5). The homogeneous branch has a nonmonotone depth profile governed by the set bits of the observation time. A separate one-path rule shows how relative velocity changes the number of distinct initial controls read (Theorem 3). The branching examples show how to transfer a predictor across a change of frame and how to compile a different summary when the frozen statistics fail (Section 5).

The second result gives a certificate for individual inputs (Section 6). The response to a proposed residual is computed on the full dependency cone. The local equations then determine whether that residual is the actual one. We use the same check for both parallel enumeration and a chronological repair procedure. Their size and depth bounds depend differently on the residual budget.

2 Preliminaries

A local rule on configurations over $\mathbb{Z}$ is iterated T times. We ask for one data bit, at site zero. The controlled models have one-sided radius-one updates and use the initial window at sites $0, \ldots, T$. In Section 6 we use a symmetric window for a general finite radius. For the frozen model the input contains $q(T + 1)$ control bits followed by $T + 1$ data bits. The control width q and the two guard circuits are fixed independently of T. The legal word lengths are $n = (q + 1)(T + 1)$ and determine T uniquely. The prediction language $L_{s,h}$ consists of the legal words whose predicted bit is one, with all other lengths rejected.

Our upper bounds use Boolean AND and OR gates of fan-in two, NOT gates, and constants. An input wire has depth zero and a NOT gate costs one layer. Gates can share outputs and connect arbitrary sites. The lower bounds in the classification apply to any Boolean gates of fan-in at most two. For a fixed rule, write $D(T)$ for the minimum De Morgan circuit depth on the legal window at time T and write $\ell(m) = \lceil \log_2 m \rceil$ for $m \geq 1$. Uniform circuit families are indexed by input length. A deterministic logspace transducer receives the length in unary and writes the complete circuit code, including its gates, wiring, and output. We use logspace uniformity throughout the statements about NC.

A coordinate is *essential* if two assignments that differ only at that coordinate give different outputs.

Essential-input bound. A depth-d circuit of fan-in at most b has at most b^d essential coordinates. For $b \geq 2$, a function with $m \geq 1$ essential coordinates therefore requires depth at least $\lceil \log_b m \rceil$.

Proof. Only input wires in the backwards cone of the output can affect it. At depth zero the cone has at most one input. At the next depth a gate has at most b predecessor cones, so induction bounds the number of input wires by b^d. In a shared circuit, counting a repeated predecessor more than once only increases the bound. □

3 Frozen-control automata

3.1 The iterate formula

Let $a_i \in \{0,1\}^q$ be a control word, $x_i \in \mathbb{F}_2$ an initial data bit, and $s, h : \{0,1\}^q \to \{0,1\}$ the stay and hop guards. The synchronous update is

$$a_i' = a_i, \quad x_i' = s(a_i)x_i + h(a_i)x_{i+1}, \tag{1}$$

where the data arithmetic is in $\mathbb{F}_2$. A backwards path ending at site k has k hops and $T - k$ stays. Put

$$e_k = \prod_{j=0}^{k-1} h\big(a_j\big), \quad w_k = \sum_{j=0}^{k} s\big(a_j\big), \tag{2}$$

where the second sum is in the natural numbers. For natural w, r, write $M(w, r)$ for the number of weak compositions of r into w parts. For $w > 0$, it is $\binom{w+r-1}{r}$, and $M(0, r)$ is one at $r = 0$ and zero otherwise.

Theorem 1 (Iterate formula). *For every initial configuration and time T, the data bit at site zero is*

$$x_0(T) = \sum_{k=0}^{T} e_k (M(w_k, T-k) \bmod 2) x_k. \tag{3}$$

The same identity holds at any site after translating the indices.

Proof. Expanding the local recurrence once per step expresses the output as a sum over backwards paths. Each hop must be enabled, contributing the factor e_k. A path visits the sites 0 through k in order. The number of stays allocated to each enabled site determines the path, including possible stays after the last hop. There are $M(w_k, T-k)$ such allocations. Their contributions add in $\mathbb{F}_2$, giving Equation 3. The Lean proof establishes the corresponding coefficient recurrence by induction on the iterate of Equation 1. The formula involves only the initial window. □

3.2 Depth classification

Write $p(T)$ for the number of ones in the binary expansion of T, and d_s, d_h for the depths of the fixed guard circuits. A guard varies if it takes both Boolean values. All the bounds below are finite, including $T = 0$.

Guard condition	Lower bound	Uniform upper bound
$h \equiv 0$	0	$1 + d_s$
$h \equiv 1$, $s \equiv 0$	0	0
h varies, or $h \equiv 1$ and s varies	$\ell(T+1)$	$O(\log(T+2))$
$s \equiv h \equiv 1$	$p(T)$	$3p(T)$

Table 1: Depth bounds at legal lengths for fixed frozen guards.

Theorem 2 (Uniform classification). *For each fixed pair of guard circuits there is one logspace-uniform, polynomial-size De Morgan family that decides the whole language and achieves the appropriate upper bound in Table 1 at every legal length. It has depth $O(\log(n+2))$ at all lengths. The lower column bounds $D(T)$ and holds even for arbitrary fan-in-two Boolean gates. Constant-depth membership is characterized by*

$$L_{s,h} \in \text{logspace-uniform } \mathrm{NC}^0 \quad \Leftrightarrow \quad h \equiv 0 \vee (h \equiv 1 \wedge s \equiv 0). \tag{4}$$

3.3 Proof of the uniform classification

Local and shift cases. For $h \equiv 0$ and $T > 0$, the answer is $s(a_0)x_0$ by idempotence. The pure shift case returns x_T. At $T = 0$ every rule returns x_0.

Varying guards. When h varies, set all data bits except x_T to zero. The surviving term in Equation 3 is $x_T \prod_{j=0}^{T-1} h(a_j)$, regardless of s. A nonconstant Boolean guard has an essential bit, as seen along a single-bit path between two words with different guard values. Each of the first T control words contributes one essential bit on this restriction, and x_T contributes one more. When $h \equiv 1$ and s varies, fix $s(a_0) = 1$ and $s(a_i) = 0$ at sites 1 through T. Then $w_k = 1$ for every k and Equation 3 is parity of all $T + 1$ data bits. Both cases give the same lower bound by essential coordinates.

The upper bound in this branch is given by the construction in Section 3.5.

Homogeneous Pascal rule. For $s \equiv h \equiv 1$, the coefficients are $\binom{T}{k} \bmod 2$. Over $\mathbb{F}_2$, the identity

$$(1+z)^T = \prod_{j:\mathrm{bit}_{j(T)}=1} \left(1 + z^{2^j}\right) \tag{5}$$

shows that precisely the binary submasks of T occur. Their $2^{p(T)}$ data bits are essential. The logarithmic lower bound is therefore $p(T)$. A balanced XOR tree reduces those leaves in $p(T)$ rounds. Each XOR can be implemented with at most three De Morgan layers. The uniform generator scans $k = 0, \ldots, T$, emits the submask wires, and serializes a regular XOR reducer with $5p(T)2^{p(T)}$ gates.

A single uniform family. The guard circuits and the selected branch are fixed. The logspace transducer writes that branch's predictor at legal lengths and a rejecting circuit at other lengths. Its polynomial-length output bounds the number of gates. The local and shift branches have a constant bound. In the varying branch the lower bound grows with T. For the homogeneous branch it grows along the times $T = 2^r - 1$. This proves the constant-depth characterization. □

The accompanying classifier tests constancy of the guards by traversing control words by prefix. Lean proves that its return value is equivalent to the branch condition. A constancy test stops at its first counterexample and can visit 2^q words in the worst case.

3.4 Exact depth in the Pascal case

Write $p = p(T)$ and $N = 2^p$. The Pascal output is parity of N inputs. With XOR available as a single binary gate, a balanced tree attains the lower bound p. With De Morgan gates the exact depth is zero at $T = 0$ and $2p + 1$ at $T > 0$.

Proof. For the upper bound, give every leaf both its bit and its negation. A node with child parities a, b computes $a \oplus b$ and its complement in two layers from $a, \neg a, b, \neg b$. For example, the positive output is

$(a \wedge \neg b) \vee (\neg a \wedge b)$ and the negative output is $(a \wedge b) \vee (\neg a \wedge \neg b)$. The p internal levels therefore add $2p$ layers to the one layer of leaf negations.

For the lower bound, set irrelevant inputs to constants and unfold the circuit into a formula of the same or smaller depth. Push negations to the leaves and simplify constants. Khrapchenko's parity bound requires at least N^2 literal leaves [2]. Pushing negations preserves leaf count and simplifying constants can only decrease it. A formula of depth at most $2p$ has at most $2^{2p} = N^2$ leaves, with equality only for a full binary tree with all leaves at depth $2p$. That tree has only AND and OR gates and hence computes a monotone function. Parity of $N \geq 2$ inputs is nonmonotone. Thus depth at most $2p$ is impossible. □

Lean verifies both per-time optima. The De Morgan profile drops from $2r + 1$ at $T = 2^r - 1$ to three at $T = 2^r$, for $r \geq 2$. The uniform classification uses the separate $3p$ generator in Table 1.

3.5 Uniform prediction circuit

Let $B(a)$ be the set of one-bit positions in the binary expansion of a. For $w > 0$, Lucas's theorem says that $M(w, r)$ is odd exactly when $B(r) \subseteq B(w + r - 1)$ [3]. Equivalently, adding r and $w - 1$ produces no carry. Indeed, if the bits of r are contained in those of the sum, subtracting r clears those bits without a borrow. The remaining bits are those of $w - 1$. Including the case $w = 0$, the coefficient test is

$$M(w, r) \bmod 2 = 1 \quad \Leftrightarrow \quad (w = 0 \wedge r = 0) \vee (w > 0 \wedge B(r) \cap B(w - 1) = \varnothing). \tag{6}$$

The hop prefix e_k decides whether a path can reach k. Once the hops are enabled, the stay count w_k suffices because the enabled sites can each absorb any number of stays. For example, with two enabled stay sites and one stay to allocate, both possible paths cancel. In Equation 6 this is the shared low bit of $r = 1$ and $w_k - 1 = 1$.

From the test to a circuit. Evaluate the guards in parallel. Compute each e_k by a balanced AND tree and each w_k by a carry-save counter. Each merge round compresses two pairs of words into one pair at constant depth, and the final addition propagates carries across only $O(\log(T + 2))$ bits. For each k the generator knows $r = T - k$. It checks Equation 6 for each candidate count $j = 0, \ldots, k + 1$ by a binary scan, and the circuit tests equality to w_k for the candidates that pass. At most one count matches. Gate the selected bit by $e_k x_k$, then take parity over the sites. This uses a fixed number of logarithmic-depth stages and polynomial size, even with a separate counter for every prefix.

The verified generator. Lean verifies a shared decoder that can handle any time u within the window. It generates the table for all such u using the Lucas bit test, and selects the rows for T with a one-hot tag. The tag implements the composition interface by routing wires according to the binary digits of T. Each parity bank reads the low bit of a carry-save counter. A logspace transducer emits the complete gate and output lists of this circuit, using recomputation to resolve references. The explicit depth bound is $104(\ell(T + 3) + 1) + \max(d_s, d_h)$. Appendix A describes the layout and the resource accounting.

4 Transported controls as a structural comparison

In this separate model the control track has its own dynamics and the data follows one path. Fix velocities $u, v \in \{0, 1\}$ and a Boolean control and data track (c_i, d_i). Define the synchronous rule

$$c_i' = c_{i+u}, \quad d_i' = c_i \wedge d_{i+v}. \tag{7}$$

The window has $2(T + 1)$ bits. With a single backwards data path, the data at site i and time T equals

$$d_{i+vT} \wedge \bigwedge_{k=0}^{T-1} c_{i+kv+(T-1-k)u}, \tag{8}$$

where the right-hand side uses initial bits and the conjunction is empty at $T = 0$.

Theorem 3 (Exact depth under transport). *Depth is zero at $T = 0$. For $T > 0$, it is exactly one if $u = v$ and exactly $\ell(T + 1)$ if $u \neq v$. The formal upper-bound construction uses a padded AND tree with at most $2m$ binary AND gates for m essential inputs. The lower bound applies to all fan-in-two Boolean circuits predicting the data bit.*

Proof. At the k-th step of the backwards path we read control at site $i + kv$ and time $T - 1 - k$. The initial site of that control bit is $i + kv + (T - 1 - k)u$, which proves Equation 8. At $T = 0$ the answer is the input wire d_i. At positive times with equal velocities all control indices coincide, leaving a conjunction of one control bit and one data bit. In general, the initial control index is $i + (T - 1)u + k(v - u)$. For unequal velocities the indices are $i, \ldots, i + T - 1$ in forward or reverse order. Every distinct factor is essential, since flipping it with all other factors set to one changes the output. Thus there are two essential coordinates in the equal case and $T + 1$ in the unequal case. For m factors, a balanced AND tree of height $\ell(m)$ attains the essential-input lower bound. Padding to the next power of two uses $2^{\ell(m)} - 1 \leq 2m$ binary gates. □

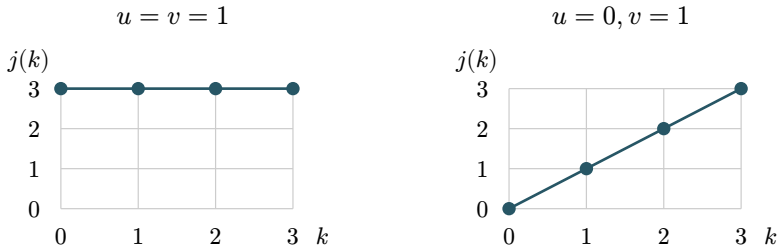


Figure 1: Initial control site $j(k)$ at backward step k for $i = 0$ and $T = 4$. Both paths end at the initial data bit d_4.

Figure 1 keeps the data path fixed and varies only u. The pair $(u, v) = (0, 0)$ is the frozen no-hop case with stay guard c_i. Changing only u to one gives logarithmic depth although the data never hops. For $(u, v) = (1, 1)$ the data and control both move, yet their equal velocity gives constant depth. For this single-path rule, the relevant count is the number of distinct initial controls encountered in the moving frame of the data path. The time-dependent framework in [1] accommodates this rule. For finite Boolean monomial systems, the link between iterate support and fixed-length dependency walks is classical [4]. On the finite prediction window, the two shifts make that support explicit.

5 Evolving controls with branching

Uniform transport. Allow both terms in Equation 1 and transport the entire control track by $a_{i(t+1)} = a_{i+1}(t)$. In the reflected moving frame $y_{j(t)} = x_{-j-t}(t)$, substitution gives

$$y_{j(t+1)} = h\big(a_{-j-1}(0)\big)y_{j(t)} + s\big(a_{-j-1}(0)\big)y_{j+1}(t). \tag{9}$$

The guards are now frozen and exchanged. The observation $x_0(T)$ is $y_{-T}(T)$, whose data window is the reversal of the original one. The frozen control window is $a_{T-1}(0), \ldots, a_0(0)$ followed by a padding word. At the last virtual site no stay can occur, so the padding is irrelevant. The two prefix statistics still work in this frame, with the roles of stay and hop exchanged. Lean verifies the coordinate change and a costed reduction with a depth-zero input rewire and identity readout. Its logspace serializer transfers the frozen upper bounds to the transported rule.

Additive control evolution. Now let the control be one bit and evolve it by $c_{i(t+1)} = c_{i(t)} + c_{i+1}(t)$, with data update $x_{i(t+1)} = c_{i(t)}x_{i(t)} + x_{i+1}(t)$. Both updates use $\mathbb{F}_2$. At $T = 2$, the coefficient of $x_1(0)$ at the origin is $(c_0(0) + c_1(0)) + c_1(0) = c_0(0)$. The initial pairs $(1, 0)$ and $(0, 1)$ have the same stay count and hop flag for this endpoint. With data $(0, 1, 0)$ they give different predictions, so no readout of that summary and the data context can predict every window. Lean checks this collision on the actual iterate. A stay can now read different guard values at the same site, making the order of controls relevant.

This rule still has a short coefficient description. Put $Q_{T(i)} = c_{i(T)} + c_{i+T}(0)$, the Pascal parity of the initial controls with its rightmost term removed. For $0 \le k < T$, put

$$P_{T,k}(i) = \prod_{j=k+1}^{T-1} \big(1 + c_{i+j}(0)\big). \tag{10}$$

Then the coefficient of $x_{i+k}(0)$ in $x_{i(T)}$ is $P_{T,k}(i)Q_{T(i)}$ for $k < T$, and one for $k = T$. Thus a single parity is shared across endpoints, and a suffix-zero test selects which endpoints receive it.

Proof. Write $b_{T,k}(i)$ for the coefficient and take it to be zero outside the window. The update gives

$$b_{T+1,k}(i) = c_{i(T)} b_{T,k}(i) + b_{T,k-1}(i+1). \tag{11}$$

Since a bit squares to itself, $c_{i(T)}Q_{T(i)} = (1 + c_{i+T}(0))c_{i(T)}$. For $k < T$, the stay term therefore has suffix factor $P_{T+1,k}(i)$, as does the shifted term because $P_{T,k-1}(i+1) = P_{T+1,k}(i)$ for $k > 0$. For $k = 0$ the shifted term is absent, but $Q_{T(i+1)}$ already vanishes whenever $P_{T+1,0}(i) = 1$: the suffix then zeros every initial control in that parity. After factoring out the suffix the sum is $c_{i(T)} + Q_{T(i+1)} = Q_{T+1}(i)$. For $k = T$ the stay term is $c_{i(T)}$ and the shifted term is $Q_{T(i+1)}$, giving the same sum with empty suffix. This also holds at $T = 0$ since $Q_0(i+1) = 0$. The all-hop term is always one. □

The verified predictor. For the first T sites, the generated circuit reduces three-bit summaries (Z, V, Q). The bit Z tests whether the block has all-zero controls. The bit V is the parity of its data bits with only zero controls to their right within the block. The bit Q collects the masked control parity. Put $\lambda_{T,j} = \binom{T}{j} \bmod 2$. Site j contributes $\big(1 + c_{j(0)}, x_{j(0)}, \lambda_{T,j}c_{j(0)}\big)$. Concatenating adjacent blocks gives the associative merge

$$(Z_L, V_L, Q_L), (Z_R, V_R, Q_R) \mapsto (Z_L Z_R, V_L Z_R + V_R, Q_L + Q_R). \tag{12}$$

The identity is $(1, 0, 0)$ and the readout is $x_{T(0)} + QV$. The input interleaves the T control-data pairs and ends with $x_{T(0)}$. The logspace generator obtains T from the odd input length and uses the Lucas submask test to hardwire the masks. Even lengths are rejected. The ordered-summary compiler pads with identities and halves the active width at each level. Lean checks the actual generated family, including copy gates, and proves a gate bound of $108(T+1)$ and, for $T \ge 1$, a depth bound of $6 + 5\ell(T)$.

Lower bound. For $T > 0$, set $c_0(0) = 1$ and all the other controls in the input to zero. Every data coefficient becomes one. All $T + 1$ data bits are then essential, giving the Lean-verified lower bound $\ell(T+1)$ for any fan-in-two Boolean circuit. Thus the minimum De Morgan depth is $\Theta(\log(T+1))$ as T grows. At time zero the prediction is a data projection. The formula also gives a sharper count on paper. With only $x_0(0) = 1$ on the data track, the output is $c_0(0) \prod_{j=1}^{T-1} \big(1 + c_{j(0)}\big)$, so all T control bits are essential too. This gives the counting bound $\ell(2T+1)$.

6 Sparse corrections along a trajectory

The preceding results exploit structure shared by every input of a fixed rule. Here the certificate concerns one trajectory and horizon. It counts the discrepancies between the actual updates and those of a fixed linear reference, evaluated on the actual state at each time.

6.1 Common cone and forced response

Fix a deterministic rule F, a homogeneous linear rule L, a track count $q \ge 1$, and a common radius bound $R \ge 1$. At horizon T, let $B_t = [-R(T-t), R(T-t)]_{\mathbb{Z}} \times \{0, \dots, q-1\}$ be the bit coordinates at state time t. Each neighborhood of a site in B_{t+1} is contained in the spatial window of B_t. Consequently the restrictions $F_t, L_t : \mathbb{F}_2^{B_t} \to \mathbb{F}_2^{B_{t+1}}$ are well defined and induction shows that the finite iteration agrees with every infinite extension of the initial word u. We observe a fixed track a_* at site zero at time T. Write $g_T(u)$ for that bit. The legal lengths $n = q(2RT+1)$ determine T. The number of state bits

is $V = q(T+1)(RT+1) = O(n^2)$, and there are $M = V - n$ update-output bits. An event (t, i, a) denotes the bit at (i, a) in B_{t+1} created by the update from time t to $t+1$.

Let $x_0 = u$, $x_{t+1} = F_t(x_t)$, and write the actual residual and the forced reference trajectory as

$$e_t^\star = x_{t+1} + L_t x_t, \quad y_0 = u, \quad y_{t+1} = L_t y_t + e_t. \tag{13}$$

All sums are in $\mathbb{F}_2$. The rectangular propagator $P_{s,t} = L_{t-1} \ldots L_s$ carries layer s to layer t, with the identity at $s = t$. Linearity and induction in Equation 13 give

$$y_t = P_{0,t} u + \sum_{s=0}^{t-1} P_{s+1,t} e_s. \tag{14}$$

Let S and D be the gate count and depth of a circuit returning **every** bit of this response. Each y_{t+1} is checked against $F_t(y_t)$, including coordinates where the proposed residual is zero. The check costs $O(V)$ gates and $O(\log(V+1))$ depth by locality and a balanced conjunction. Write $v(u, e)$ for its result. If $e = e^\star$, Equation 13 gives $y = x$, so the check passes. Conversely a passing response and the actual trajectory share their initial state and satisfy the same update at every coordinate. Induction gives $y = x$ and Equation 13 gives $e = e^\star$. Hence

$$v(u, e) = 1 \Leftrightarrow e = e^\star. \tag{15}$$

6.2 Parallel enumeration and chronological repair

Theorem 4 (Enumeration certificate). *For an input-independent list of $N \geq 1$ candidate residuals, a circuit outputs $(A, A \wedge g_T(u))$, where A tests whether $e^\star$ occurs in the list. Its size is $O(N(S+V+1))$ and its depth is $D + O(\log(V+1) + \log(N+1))$.*

Proof. Substitute each candidate into its own copy of the response. Take the OR of all checks $v(u, e)$ for A, and the OR of all checks ANDed with their candidate observation for the second output. Equation Equation 15 identifies the passing candidates and their common observation. Substitution uses constant wires. The copies run in parallel and the two balanced OR trees add $O(\log(N+1))$ depth. □

A residual of weight at most k is encoded by at most k event addresses. Each support has a sorted listing padded to length k with a null symbol, so the number of distinct candidates is $\sum_{j=0}^{\min(k,M)} \binom{M}{j} \leq (M+1)^k$. A generator may emit all such tuples with repetitions, interpreting each as a set. For fixed k, the counters occupy $O(\log(n+1))$ space. If the allowed events are a fixed set J_T of K distinct addresses, assume that K and the indexed addresses can be computed in logspace. Use the 2^K bit assignments on J_T and zeros elsewhere. The assignment counter occupies $O(\log(n+1))$ space when $K = O(\log(n+1))$. In both cases a generator orders states and events by time, site and track. It substitutes each candidate constant for a correction input, scans the response gate code, relocates its internal wire indices by the copy offset, and emits the local checks and balanced reductions in indexed blocks. Polynomial gate counts give logarithmic-width counters for all offsets and references. At illegal lengths both outputs reject. Thus a uniform polynomial-size response of depth $O(\log(n+1))$ gives two logspace-uniform NC^1 predicates, applicability and gated prediction, in both regimes.

Theorem 5 (Chronological repair certificate). *Partition the update-output bits into singletons in chronological order or into whole update layers in time order. For a budget k, a circuit outputs $(A, A \wedge g_T(u))$, with A testing whether at most k blocks contain a nonzero actual residual bit. Its size is $O((k+1)(S+V+1))$ and its depth is $O((k+1)(D+\log(V+1)))$.*

Proof. Start at $e = 0$. In a round compute the response y, and form the mismatch $m_t = F_t(y_t) + y_{t+1}$. Find the earliest block containing a nonzero mismatch and XOR the mismatch into e on that block alone. With no mismatch, the round leaves e unchanged.

At the start of a round the current residual agrees with $e^\star$ on a prefix ending at the last repaired block and is zero later. The empty prefix is allowed initially. Every earlier update layer than the first unrepaired nonzero block now has the actual residual, so its trajectory agrees with x through the predecessor state

of that block. At its update time the mismatch is exactly $e_t^\star + e_t$, because $F_t(y_t) = x_{t+1}$ and $y_{t+1} = L_t x_t + e_t$. This also shows that all earlier blocks in the same layer have zero mismatch. Thus the chosen block is the first missing nonzero block of $e^\star$, and the round restores it without changing other blocks. If none is missing, the full check already passes. After k rounds apply Equation 15 once more and gate the final observation by this check.

Compute the mismatch bits and each block's nonzero flag in parallel. Place the flags at the leaves of a balanced binary tree and compute each subtree's OR bottom-up. Then propagate prefixes downward from a zero prefix at the root. The left child inherits the parent's prefix. The right child ORs that prefix with the left subtree's OR. At a leaf the prefix is the OR of all preceding flags. Select a block exactly when its flag is one and its prefix is zero. The two passes use $O(V)$ gates and $O(\log(V+1))$ depth. Each residual-bit update adds constant depth. There are k repair stages and one final response and check, which gives the claimed bounds. □

The repair layout is logspace uniform for a uniform response and a logspace-computable, polynomially bounded round budget. The generator loops over stages and blocks, reuses the response serializer with relocated inputs, and wires the new residual bits to the next stage. Tree references are computed from level and node indices. This replaces the enumeration exponent depending on k by a multiplicative factor $k+1$. With a logarithmic-depth response it gives uniform NC^1 for constant k and uniform NC^2 for $k = O(\log(n+1))$. With whole-layer blocks, an update may contain any number of residual bits. With singleton blocks the bound is on total bit weight. At $T = 0$ there are no events, the empty check accepts and the observation is an initial bit in both constructions.

6.3 A uniform response for any fixed homogeneous linear rule

Theorem 6 (Homogeneous linear response). *Every fixed q-track radius-R homogeneous linear rule over $\mathbb{F}_2$ admits a logspace-uniform full forced response on the common cone with $O(V^2)$ gates and depth $O(\log(V+1))$.*

Proof. Write $(Lx)_i = \sum_{\delta=-R}^{R} A_\delta x_{i+\delta}$ with fixed $q \times q$ matrices. Let $H(X) = \sum_{\delta=-R}^{R} A_\delta X^{\delta+R}$. The coefficient from site j to site i after r steps is the appropriate matrix entry of $[X^{j-i+Rr}]H(X)^r$, with out-of-range exponents contributing zero. Cone closure identifies these with the finite propagator coefficients. They are coefficients of the rational matrix series

$$\sum_{r \geq 0} Y^r H(X)^r = (I - YH(X))^{-1}. \tag{16}$$

Each entry has polynomial numerator U and common denominator $Q = \det(I - YH(X))$ with $Q(0,0) = 1$.

For $a, b \in \{0,1\}$, let $\Lambda_{a,b}$ take the coefficients at indices $(2i+a, 2j+b)$ of a series and place them at (i,j). This is the binary section operator used in automatic coefficient algorithms [5]. Over $\mathbb{F}_2$ we have $Q(X,Y)^2 = Q(X^2, Y^2)$. The inverse of $Q(X^2, Y^2)$ has only even exponents in both variables, so the section operator passes it through with those exponents halved. Applying this to $\frac{UQ}{Q(X^2,Y^2)}$ gives

$$\Lambda_{a,b}\left(\frac{U}{Q}\right) = \Lambda_{a,b}\frac{UQ}{Q}. \tag{17}$$

For each variable, bound the numerator degree by the larger of the degrees of the original numerators and Q. Multiplication by Q and halving the exponents preserve these bounds. There are finitely many numerators over $\mathbb{F}_2$ in the fixed degree box. To obtain the coefficient at a given pair of indices, start at the relevant numerator and read the two binary indices least significant digits first, padding the shorter one with zeros. Replace the numerator by $\Lambda_{a,b}(UQ)$ at each digit pair. At the end, return the constant coefficient of the current numerator, since $Q(0,0) = 1$. These fixed transitions and logarithmic-width index counters give a logspace coefficient generator.

For each output bit, list all initial and earlier forcing bits in Equation 14. Use its generated coefficient to wire either the source bit or zero into a balanced XOR tree. There are at most V leaves per output and V outputs. XOR has a fixed De Morgan realization. The same time, site and track counters give the source and output addresses. Padding the trees to a common power of two makes their wire offsets computable by integer arithmetic on logarithmic-width counters. This emits the complete response circuit in logspace. □

The homogeneous response theorem supplies the circuit hypothesis of both certificates for the stated lattice model. For a time-dependent or position-dependent linear background, Equation 14 still holds, and a shallow uniform response must be supplied separately.

6.4 Two concrete backgrounds

For the one-track Pascal reference $(Lx)_i = x_i + x_{i+1}$, an interval of r steps uses exactly the offsets whose set bits occur in r. This is the binary coefficient test from Section 3.4. The actual OR rule $(Fx)_i = x_i \vee x_{i+1}$ has residual $x_i x_{i+1}$ on each update. Its independently known apex is $\bigvee_{j=0}^{T} x_j$. This supplies a check of the gated output on accepted inputs.

For a reversible nonlinear example, use tracks (a, b, c) and cycle the local triples $110 \to 101 \to 011 \to 110$, fixing all other triples. Then stream a right, b left and keep c stationary. The local permutation and the streaming map are invertible and preserve the total number of ones on finite-support configurations. The cycle fixes the three basis triples and moves 110, which proves nonlinearity. Take free streaming for L. Each forced output is the parity of the initial bit and all impulses on its characteristic. For example,

$$a_i(t) = a_{i-t}(0) + \sum_{s=0}^{t-1} e^{a}_{s,i-(t-1-s)}. \tag{18}$$

The b characteristic has the opposite direction and the c characteristic stays at i. Each output uses at most $T + 1$ terms. The streaming response therefore has size $O(V(T + 1))$ and depth $O(\log(T + 2))$.

Put ones at a_0, c_0 and c_{-h} for any integer $h \geq 1$, with all other bits zero, and query $a_0(2h)$. At time zero the local state 101 at the origin becomes 011, sending b to the left. At time h the state 011 at site $-h$ becomes 110, creating a right-moving a. It reaches the origin at time $2h$, with no other nontrivial collision before that observation. The four global residual bits are

$$e^{a}_{0,1}, \quad e^{b}_{0,-1}, \quad e^{a}_{h,-h+1}, \quad e^{c}_{h,-h}. \tag{19}$$

The last lies outside the common cone at state time $h + 1$, whose radius is $h - 1$. The other three lie inside. Removing either stationary particle makes the queried bit zero, so both interactions are relevant to the observed return. This gives admissible interacting trajectories at arbitrarily large horizons. The certificate supplies bounds on the admissible input language and the gated observation.

7 Related work

Pascal parity and fast prediction. The odd-entry count $2^{p(T)}$ is classical. Rowland attributes the binary count to Glaisher and derives it from Lucas's theorem [3]. Moore uses Pascal coefficients to predict homogeneous additive rules, obtaining logarithmic depth with binary gates and logspace-uniform ACC^0 with unbounded modular sums [1]. Robison gives a serial method for computing an individual coefficient from one initial seed [6]. The per-time optimum in Section 3.4 follows from classical parity complexity. The verified uniform family realizes the Pascal profile within a constant factor.

Dependent linear tracks and the coefficient formula. In [1], Lemma 1 composes spacetime transfer coefficients by dividing the time interval. Theorem 1 applies that algorithm to quasidirect products. Examples 1 and 3 cover binary data tracks controlled by another track and triangular linear tracks such as our additive-control example. With balanced XOR trees for the modular sums, the general method

gives an $O(\log^2(T+2))$ bound for our binary tracks. Time invariance and the one-sided stencil reduce the frozen coefficients to prefix statistics. In §3, Salo studies the sitewise choice between identity and right-neighbor XOR on a finite ring [7]. That example uses the local maps of $s \equiv 1$, $h(a) = a$ and concerns surjectivity and periodicity. Here the control word is part of the input and the output is one finite-window prediction bit.

Parallel prefix circuits are classical [8]. Balanced products, carry-save counting, and coefficient tests give the frozen predictor's $O(\log(T+2))$ bound. The coefficient in Equation 3 also has a classical matrix analogue. The one-step data map is upper bidiagonal on the relevant finite window. Higham's Theorem 6.1 for functions of such matrices gives the product of intervening off-diagonal entries times a divided difference in the diagonal entries [9]. For the monomial z^T this divided difference is a complete homogeneous polynomial of degree $T-k$. On zero-one diagonal entries it counts the same weak compositions. The polynomial identity can be specialized to integer zero-one entries before reducing modulo two, with no division in $\mathbb{F}_2$.

Linear references and sparse trajectories. Moore's decomposition also gives a defect prediction example [1]. Automatic spacetime diagrams for scalar linear automata were studied by Rowland and Yassawi [10]. The binary section argument in Theorem 6 applies to the entries of a fixed matrix transfer series and gives the wiring of the complete forced response. Small SAT backdoors distinguish exploiting a given variable set from finding one [11]. Our candidate locations are addresses of residual bits on a trajectory. The linear components optimized by Boyar, Find and Peralta are parts of a fixed circuit [12], whereas the nonzero residuals here can vary with the input. For freezing networks, bounded alphabet, degree and treewidth support fast parallel prediction [13]. The parameter here counts discrepancy bits or layers relative to the chosen reference, even when the reference changes on every step.

Mechanization and the meaning of frozen. Silváši and Tomášek formalized bounded grids and computable cellular automata in Lean [14]. Their work supplies precedent for machine-checked cellular automata. The present artifact checks a rule-family depth classification and the actual circuit generator. The freezing model of Ollinger and Theyssier imposes a monotone order on each whole cell state [15]. In Equation 1 only the control word is fixed, and the data at a site can cycle. The contribution here is the executable four-way classification, the matching logarithmic bounds for varying guards, and the proof that one logspace generator per rule achieves the stated upper bounds across all lengths. The comparisons above leave priority for this combination open.

8 Mechanization and scope

The window interface separates infinite dynamics from finite inputs. `rawSample` reads the window from a configuration, and `decodeRaw` extends an arbitrary legal window to an infinite configuration. Lean proves that sampling this extension recovers the supplied bits. Correctness on sampled configurations therefore gives correctness on every legal input. Lifting from time to input length selects that same predictor at legal lengths and a rejecting circuit at the others.

The uniformity proof generates the code of the circuit used for the depth bound, including the variable-length lists of gates and output references. The reusable code combinators verify reference bounds and relocation for parallel banks and serial composition. These obligations allow a generator to recover individual references by recomputation in logarithmic space. The classification theorem uses the selected circuit family as one witness for correctness, size, uniformity and the per-time bound. The transport theorem establishes optimal depth separately at each window length. The ordered-summary compiler also realizes the additive and OR predictors. It charges initialization, merging, and readout, and preserves block order while changing the parenthesization.

Each frozen control word selects one of four local data maps 0, x_i, x_{i+1} and $x_i + x_{i+1}$. Once the controls are fixed, the data dynamics is one-sided and linear. The input is an explicit window with length linear in T, and the circuit can wire across arbitrary sites. Local dynamics and locality of the predictor's wiring

are separate assumptions. The results here use unrestricted wiring and the full explicit input domain of each model.

The accompanying Lean development also formalizes three general criteria for prediction problems. A compiles resource-bounded composition circuits into a predictor. B transfers P-hardness from circuit evaluation given a logspace simulation witness. Exclusion from uniform NC via B assumes $\mathrm{P} \neq \mathrm{NC}$ and is a complexity-class statement. C gives an unconditional depth lower bound from essential inputs. At fixed fan-in, dimension, radius, and bit width, a polynomial-volume light cone limits the **counting bound** from C to $O(\log(T+2))$. Actual prediction depth can exceed this bound. The finite summary recognizer checks local equations for a supplied evaluator and template. Its conclusion uses a proof connecting that evaluator to the prediction task. A collision rules out its candidate summary, and the width theorem concerns an independent summary at a fixed cut. Both conclusions concern the specified summary interface. In the frozen family, the executable classifier selects the depth branch by testing guard constancy on a fixed finite alphabet.

Lean checks the finite cone, reconstruction, verification, and enumeration, including gate and depth bounds, the bit length of the serialized circuit, and uniformity under the layout hypotheses. It verifies both chronological repair circuits, including their acceptance and gated value, their size and depth bounds, and their exact logspace serializer. Under the uniform layout and block hypotheses, fixed and logarithmic round budgets give uniform NC^1 and NC^2, respectively. The Pascal and free streaming responses have checked circuits and logspace generators. The OR and collision predictors have end-to-end uniformity proofs. Lean verifies the direct disjunction for OR and the exact collision thresholds of three bit rounds and two layer rounds. The text proves the general homogeneous coefficient generator using section operators. See Appendix C.

9 Conclusion

For frozen controls, guard constancy decides the constant-depth cases, and prefix statistics give a uniform logarithmic-depth predictor for varying guards. The Pascal depth is governed by the set bits of the observation time. With branching, uniform transport preserves the frozen summary after a change of frame, while additive control evolution needs a different summary even with all hops enabled. In the latter example, the induction closes because the all-hop coefficient is fixed and the control evolves by Pascal convolution. It remains to determine when a comparable coefficient recurrence closes if the evolving control also gates the hops.

For trajectory-dependent prediction, complete local verification lets a shallow linear response handle sparse nonlinear corrections. Enumeration exploits a small set of possible positions, while chronological repair uses a size bound polynomial in the cone volume and the correction budget. The reversible collision family illustrates that an admissible trajectory can contain causally linked interactions.

Artifact availability

The Lean and Typst sources and the build instructions are in the project repository. Use the repository snapshot accompanying this PDF for the declarations listed in Appendix B. The repository documents the pinned Typst and font inputs for rebuilding all four editions. Run `./scripts/check.sh` for the strict Lean build, import coverage check, transitive axiom audit, and regressions. The repository's verification guide separates build evidence from review of the model assumptions. Appendix B lists the declarations behind the main results.

A Generator layout and depth budget

Put $m = T + 1$ and $B = \ell(T+3) + 1$. The register stores the time tag τ, the stay and hop flags, and the data in four blocks of width m. The tag starts at index zero. Reading the binary digits of T from most significant to least, each stage doubles the index and increments it when the digit is one. These operations route wires with zero fill.

At site k, the decoder enumerates $0 \le u < m$ and $0 \le j \le k+1$. For a row with $u \ge k$ and odd $M(j, u-k)$ it emits the test $(w_k = j) \wedge \tau_u$. All other rows are zero. Parity over j, then u, selects the coefficient. The decoder multiplies by the hop prefix and data and takes parity over k.

The normalized counter tree costs at most $2 + 14b$ layers, including initialization and copying in each merge round, for a counter of word width $b = \ell(a+1)$ on a bits. The finishing addition costs at most $1 + 8b$. Together they cost at most $22(b+1)$. The counts therefore cost at most $22B$, and taking the low bit of a counter for parity is bounded by $23B$ including the empty case. The depth proof charges $5B$ for a row test, from the literal comparisons, conjunction and time tag. The hop conjunction, bounded by $5B$, runs alongside the lookup. The decoder budget is $22B + 5B + 3 \cdot 23B + 2 = 96B + 2 \le 98B$. Register initialization costs at most two layers with normalization. There are at most $2\ell(T+1)$ subsequent register stages of at most two layers each, also counting normalization. This gives at most $6B$ for the register path and a total of $104B + \max(d_s, d_h)$ including guards.

The table has $m^2 \frac{m+3}{2}$ rows and there are $O(m^2)$ counters of at most $m+1$ input bits. The regular layouts give $O(m^3 \log^3(m+1))$ gates for fixed guards. A gate record is indexed by its stage, block, bit lane and position in a local template. For parallel banks the serializer counts earlier blocks to obtain a wire offset. In a chain of width W, identity gates $y \wedge y$ give the stage outputs consecutive indices. After G gates on N primary inputs, output slot a of that stage is wire $N + G - W + a$. This lets the serializer recompute references with counters of logarithmic width and fixed nesting depth. It streams the gate and output lists from the unary length.

B Lean correspondence

The paths below are relative to `PredictionDepth/`.

Result	Lean module and declaration
Frozen iterate	`FrozenPascal/Semantics.lean` · `iterate_data`
Window interface	`FrozenPascal/GuardRules.lean` · `rawSample_decodeRaw`
Executable classifier	`FrozenPascal/Classification.lean` · `classify`, `classify_eq_iff`

Uniform classification	`FrozenPascal/UniformClassification.lean`
	`uniform_depth_classification`
	`predictionLanguage_mem_uniformNC_zero_iff`

Uniform predictor	`FrozenPascal/UniformPrediction.lean`
	`uniformGuardedPrediction_code_logspace`
	`uniformGuardedPrediction_depth`

Code composition	`Complexity/CodeLogspace.lean` · `compCode_logspace`

Pascal support and XOR depth	`FrozenPascal/HomogeneousPascal.lean`
	`pascalSupport_card`
	`homogeneousNativeXorPrediction_depth`

Exact Pascal De Morgan depth	`FrozenPascal/PascalExactDepth.lean` `homogeneous_prediction_deMorgan_exact_depth`

Transported control	`TransportedControl.lean`
	`iterate_data_true_iff`, `essential_prediction_iff`
	`exact_prediction_depth`, `constant_depth_iff`

Branching under transport	`TransportedBranching.lean` `reduction`, `prediction_correct`, `prediction_uniform`
Ordered-summary compiler	`Complexity/BlockSummary.lean` `compile_eval`, `compile_depth`, `compile_size`, `compile_code_logspace`
Summary recognition and limits	`BlockSummary.lean` `recognize?`, `findCollision?_sound`, `width_lower_bound`
OR instance	`ORAutomaton/Summary.lean` · `summaryFamily_uniform`
Additive-control coefficients	`AdditiveBranching.lean` · `trimmedParity_eq_sum`, `iterate_data`

Additive circuit bounds	`AdditiveBranching/Summary.lean`
	`prediction_depth`, `prediction_size`, `predictionFamily_uniform`
	`AdditiveBranching/LowerBound.lean` · `prediction_depth_lower_bound`

Additive collision	`AdditiveBranching/Summary.lean` `frozenEndpointCollision`, `frozenEndpoint_noReadout`

C Sparse correction declarations

Paths are relative to `PredictionDepth/SparseCorrections/`.

Result	Lean module and declaration

Common cone and verifier	`Semantics.lean`
	`Cone.finiteTrajectory_eq_restrict_iterate`
	`valid_iff_eq_actualResidual`

Propagator formula	`Reconstruction.lean` `forcedTrajectory_eq_propagator_sum`

Enumeration compiler	`Compiler.lean`
	`compile_semantics`
	`compile_depth`, `compile_size`
	`compile_code_logspace`

Serialized description length	`Description.lean` `compile_description_length_le`

Uniform enumeration	`Certificate.lean`
	`Correct.unknown_mem_uniformNC_one`
	`Correct.scheduled_mem_uniformNC_one`

Chronological bit repair	`Repair.lean` `repair_accept_iff`
Chronological layer repair	`LayerRepair.lean` `repairLayer_accept_iff`

Parallel repair circuits	`RepairCircuit.lean`
	`compileBits_correct`, `compileLayers_correct`
	`compile_size`, `compile_depth`
	`RepairCode.lean`, `compile_code_logspace`

Uniform repair and thresholds	`RepairUniform.lean`
	`fixedLanguage_mem_uniformNC_one`
	`logLanguage_mem_uniformNC_two`
	`RepairExamples.lean`, `example_repair_thresholds`

Streaming response	`Streaming.lean`
	`forcedTrajectory_eq_characteristic`
	`StreamingCircuit.lean`
	`circuit_correct`, `circuit_code_logspace`

Forced Pascal response	`PascalResponse.lean`
	`circuit_correct`, `circuit_code_logspace`
	`circuit_size_quadratic`, `circuit_depth_logarithmic`

Uniform OR prediction	`PascalOR.lean` `orPredictor_correct`, `orPrediction_mem_uniformNC_one`

Collision example	`Collision.lean`
	`step_reversible`, `returning_observation`
	`exampleResidual_card`

OR and empty-horizon fixtures	`CompilerExamples.lean`
	`ORFixture.oneCorrection`, `ORFixture.code_agrees`
	`emptyHorizon_predicts`

Uniform collision prediction	`CollisionCertificate.lean`
	`sparsePrediction_mem_uniformNC_one`
	`example_accepts_and_predicts`

References

[1] C. Moore, "Predicting Nonlinear Cellular Automata Quickly by Decomposing Them into Linear Ones," *Physica D: Nonlinear Phenomena*, vol. 111, pp. 27–41, 1998, doi: 10.1016/S0167-2789(97)80003-6.

[2] V. M. Khrapchenko, "Complexity of the Realization of a Linear Function in the Class of Π-Circuits," *Mathematical Notes*, vol. 9, no. 1, pp. 21–23, 1971, doi: 10.1007/BF01405045.

[3] E. Rowland, "The Number of Nonzero Binomial Coefficients Modulo p^α," *Journal of Combinatorics and Number Theory*, vol. 3, pp. 15–25, 2011, [Online]. Available: https://arxiv.org/abs/1001.1783

[4] O. Colón-Reyes, R. Laubenbacher, and B. Pareigis, "Boolean Monomial Dynamical Systems," *Annals of Combinatorics*, vol. 8, no. 4, pp. 425–439, 2005, doi: 10.1007/s00026-004-0230-6.

[5] B. Adamczewski, A. Bostan, and X. Caruso, "A Sharper Multivariate Christol's Theorem with Applications to Diagonals and Hadamard Products." [Online]. Available: https://arxiv.org/abs/2306.02640

[6] A. D. Robison, "Fast Computation of Additive Cellular Automata," *Complex Systems*, vol. 1, no. 1, pp. 211–216, 1987, [Online]. Available: https://www.complex-systems.com/abstracts/v01_i01_a15/

[7] V. Salo, "Realization Problems for Nonuniform Cellular Automata," *Theoretical Computer Science*, vol. 559, pp. 91–107, 2014, doi: 10.1016/j.tcs.2014.07.031.

[8] R. E. Ladner and M. J. Fischer, "Parallel Prefix Computation," *Journal of the ACM*, vol. 27, no. 4, pp. 831–838, 1980, doi: 10.1145/322217.322232.

[9] N. J. Higham, "The Power of Bidiagonal Matrices," *Electronic Journal of Linear Algebra*, vol. 40, pp. 453–474, 2024, doi: 10.13001/ela.2024.8297.

[10] E. Rowland and R. Yassawi, "Automaticity and Invariant Measures of Linear Cellular Automata," *Canadian Journal of Mathematics*, vol. 72, no. 6, pp. 1691–1726, 2020, doi: 10.4153/S0008414X19000488.

[11] R. Williams, C. P. Gomes, and B. Selman, "Backdoors to Typical Case Complexity," in *Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence*, 2003, pp. 1173–1177. [Online]. Available: https://ijcai.org/proceedings/2003/167

[12] J. Boyar, M. G. Find, and R. Peralta, "Small Low-Depth Circuits for Cryptographic Applications," *Cryptography and Communications*, vol. 11, pp. 109–127, 2019, doi: 10.1007/s12095-018-0296-3.

[13] E. Goles, P. Montealegre, M. Ríos-Wilson, and G. Theyssier, "On the Parameterized Complexity of Freezing Dynamics," *Advances in Applied Mathematics*, vol. 157, p. 102706, 2024, doi: 10.1016/j.aam.2024.102706.

[14] F. Silváši and M. Tomášek, "Lean Formalization of Bounded Grids and Computable Cellular Automata Defined Thereover," *Science of Computer Programming*, vol. 195, p. 102471, 2020, doi: 10.1016/j.scico.2020.102471.

[15] N. Ollinger and G. Theyssier, "Freezing, Bounded-Change and Convergent Cellular Automata," *Discrete Mathematics and Theoretical Computer Science*, vol. 24, no. 1, 2022, doi: 10.46298/dmtcs.5734.